

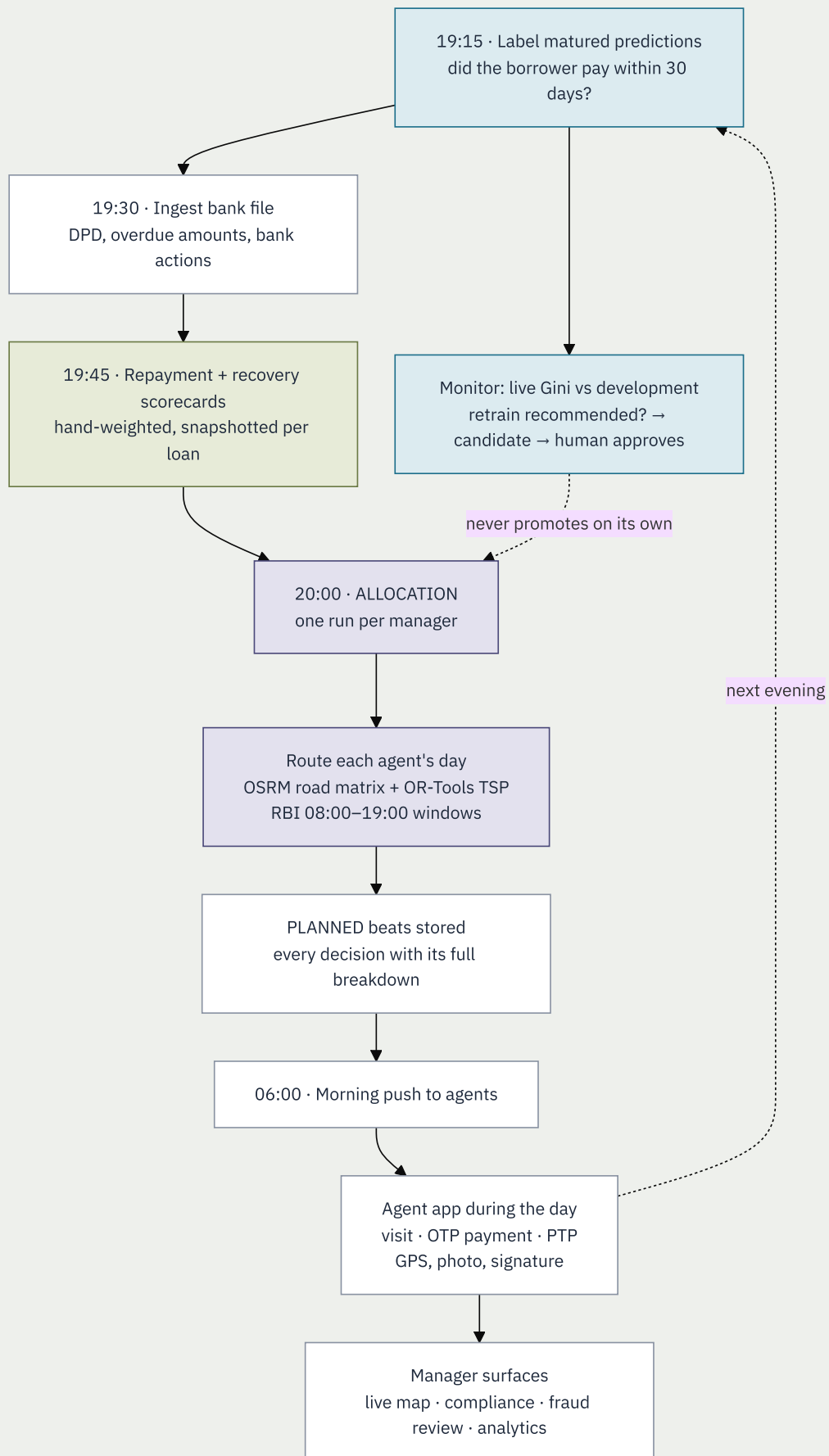
How TIQCollect decides who visits whom

Three diagrams: the whole nightly pipeline, the allocation decision opened up, and where the Empirical Bayes agent estimate fits. Each is followed by what to say out loud.

-  Hard rule — cannot be overridden
-  Trained ML model
-  Hand-weighted or statistical estimate
-  Optimisation solver

1 • The nightly pipeline, end to end

Every evening the system turns today's bank file and today's field results into tomorrow's plan. Times are IST, from `workers/celery_app.py` .

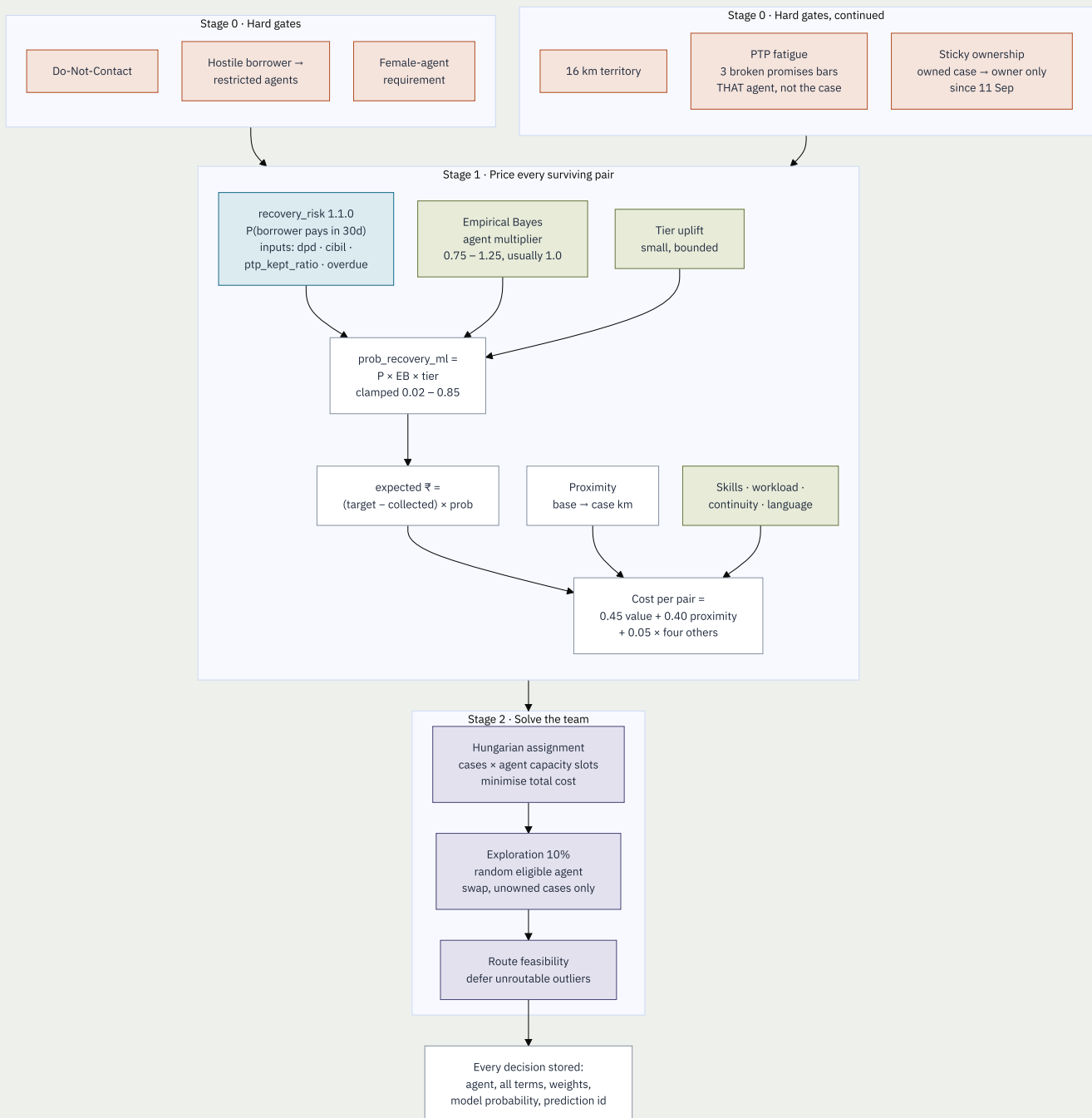


WHAT TO SAY

"Each night, four things happen in order. We score yesterday's predictions against what actually happened. We take in the bank's file. We refresh the hand-written scorecards. Then the allocator plans tomorrow and the router sequences each agent's day. Agents get it at six in the morning. Everything they do during the day feeds back into the next night's run."

2 · Inside the allocation — where every number comes from

This is the box marked ALLOCATION above, opened up. Read it left to right: rules first, then pricing, then the solve. Nothing on the right can undo anything on the left.



WHAT TO SAY — "WHY ISN'T THE MODEL THE ALLOCATOR?"

"The model answers one question: will this borrower pay? That's a fact about a person. The allocator answers a different question: given thirty agents, nine hundred cases, fifteen slots each, distances and legal constraints — who goes where? That's a team-wide optimisation. No per-borrower model can answer it."

"So the model prices each case — it decides how much a case is worth — and that's the single biggest term in the decision. Measured: before the model that term had 12% of the influence distance had. After the model, it's 99%. Equal partners. The model didn't get one small input; it took over half the decision."

"What it deliberately does not do is override a compliance rule or pick the agent. Rules are yes/no, never 'probably fine'. And the agent choice has to be explainable to a manager and a regulator — every decision is stored with its full arithmetic."

3 • Empirical Bayes — the agent-side estimate

The model knows nothing about the agent. The Empirical Bayes adjuster (`ml/empirical_bayes.py`) is where the *agent's* track record enters, and it is built to be honest about small samples.

The problem it solves

Agent Ravi has handled three personal-loan cases in the 61–90 DPD bucket and recovered on all three. Agent Meena has handled forty in the same segment at 62%. Is Ravi the better agent? Three cases prove nothing. A naïve system would rank him first and hand him the hardest work.

How shrinkage works

Instead of trusting the raw rate, we **pull it towards the segment average**, and how far we pull depends on how much evidence there is.

```
segment s      = (loan type × DPD bucket)          e.g. (PERSONAL, 61–90)
μ_s           = the segment's average recovery rate across ALL agents
w̄_js          = agent j's own raw rate in segment s, over n cases
k             = 10   (the "prior weight" — how many cases the average counts for)

shrunk rate ŵ_js = ( n / (n+k) ) × w̄_js + ( k / (n+k) ) × μ_s

multiplier α_js = clamp( ŵ_js / μ_s , 0.75 , 1.25 )
              α_js = 1.0 exactly when n < 5
```

AGENT	N	RAW RATE	WEIGHT ON OWN RECORD	SHRUNK RATE (M = 0.60)	MULTIPLIER
Ravi	3	1.00	3/13 = 23%	0.692	1.00 (n < 5)
Meena	40	0.62	40/50 = 80%	0.616	1.03
Arun	25	0.40	25/35 = 71%	0.457	0.76

Ravi's perfect three cases shrink to 0.69 and then fall to the 1.0 floor because five observations is the minimum to say anything. Meena's forty cases are believed almost at face value. Arun's genuinely weak record does move his multiplier — and it is capped so no one agent can be written off.

Where it goes

```
prob_recovery_ml = clamp( P_borrower × α_agent × tier_uplift , 0.02 , 0.85 )
                      ↑ model      ↑ Empirical Bayes  ↑ small bounded bonus
```

A calibrated **base rate** from the model, modulated by a **bounded relative effect** for the agent. It is a ratio centred on 1.0, not a second probability — that matters, because the 3 Sep defect was two estimates of the same quantity multiplied together, counting skill

twice. Here the two are different quantities: one is about the borrower, one is about the agent.

Three properties to name

Point-in-time

Fitted `as_of` a date, on a 180-day payment window ending that day. Payments are append-only with a date, so the estimate can be rebuilt honestly for any past day — which loan DPD cannot.

Segment-aware

Grouped by loan type × DPD bucket, so an agent who is strong on 0–30 DPD auto loans is not assumed strong on 90+ personal loans.

Bounded

0.75 to 1.25. An agent can at most move a case's expected value by a quarter either way. The borrower's probability stays the anchor.

WHAT TO SAY

"Agent skill enters through a Bayesian shrinkage estimate. An agent's recovery rate on a segment is pulled towards the segment average, and the pull is strong when they've handled few cases and weak when they've handled many. Under five cases it says nothing. It's bounded to plus or minus twenty-five percent, so it adjusts the borrower's probability, it never replaces it."

Say this before someone finds it: "On the live book today, 297 of 315 agent-segment cells are below the five-case threshold, so the multiplier is 1.0 almost everywhere and the allocation is effectively borrower-driven. That's correct behaviour on thin data, and it's exactly why 10% of assignments are randomised — to build the unbiased agent data this estimate needs."

The one-paragraph version

"Rules first, then the model prices the case, then a statistical estimate adjusts for the agent, then a solver arranges the whole team, then the router sequences each day on real roads inside legal hours. Every step is stored with its reasoning. The trained model is the biggest single term in the decision, and it is deliberately not the whole decision — because compliance must be rules, agent choice must be explainable, and a model that learned agent choice from our own history would learn our bias."

Sources: `services/global_allocator.py` (weights, `_prob_recovery_ml` , gates),
`ml/empirical_bayes.py` (shrinkage formula, k = 10, threshold 5),
`ml/artifacts/recovery_risk/1.1.0` (model), `workers/celery_app.py` (schedule).
Objective weights sum to 1.05 by design; read them as ratios.